

SecTest Generator: An AI-Powered Security Regression Test Generator for Source Code and Android APK Analysis

SecTest Generator: An AI-Powered Security Regression Test Generator...

Hiba Chagdaly^a, Sara Jamiri^a, Malak Lahnine^a, Fatimaezzahra Ennassiri^a,
Mohammed Iachgar^{b,*}

^a*National School of Applied Sciences, Cadi Ayyad University, Marrakesh, Morocco*

^b*National School of Applied Sciences, Cadi Ayyad University, Marrakesh, Morocco*

Abstract

The growing complexity of software systems and mobile applications has significantly increased the risk of security vulnerabilities that, if left undetected, can lead to serious data breaches and system compromises. This paper presents **SecTest Generator**, a DevSecOps platform designed to automatically detect security vulnerabilities in Java, Python, JavaScript source code and Android APK files, generate JUnit regression tests, and integrate a Security Gate into CI/CD pipelines. The platform combines a static analysis engine based on 46 security rules with an AI module powered by Ollama (llama3.2), enabling automatic prioritization of findings, mapping to MASVS/MASTG controls, and context-aware test generation. APK analysis leverages APKTool for manifest decompilation, enabling precise detection of exported components, dangerous permissions, and cleartext traffic. Preliminary evaluations demonstrate that the platform detects up to 19 vulnerabilities in known vulnerable applications such as InsecureBankv2, generating actionable JUnit tests and MASVS-aligned recommendations. SecTest Generator contributes to improved security posture in DevSecOps workflows and supports global efforts in mobile application security testing.

*Encadrant

Keywords: Security Testing, Regression Tests, DevSecOps, Android Security, MASVS, Static Analysis, Ollama AI, Spring Boot

1. Motivation and Significance

Security vulnerabilities in software systems represent one of the most critical challenges in modern software development. According to recent reports, the average cost of a data breach reached \$4.45 million in 2023 [1], with a significant
5 portion attributable to known vulnerabilities that could have been detected through automated static analysis.

The DevSecOps paradigm advocates for integrating security practices throughout the software development lifecycle [2]. However, many teams still rely on manual security reviews or expensive commercial tools that lack integration with
10 modern CI/CD pipelines. Furthermore, the proliferation of Android applications introduces additional attack surfaces, particularly through misconfigured components, dangerous permissions, and insecure communication channels.

SecTest Generator addresses these challenges by providing a unified platform that: (1) performs static analysis of source code across multiple languages, (2)
15 analyzes Android APK files using decompilation techniques, (3) generates executable JUnit regression tests to prevent vulnerability regression, (4) integrates an AI module for intelligent prioritization and MASVS mapping, and (5) enforces a Security Gate that blocks deployment when critical vulnerabilities are detected.

20 Unlike existing tools such as SonarQube [3] that focus primarily on code quality, SecTest Generator is specifically designed for security regression testing with a strong emphasis on Android mobile security standards (MASVS/MASTG) [4].

2. Software Description

2.1. Software Architecture

25 SecTest Generator is built on a three-tier architecture leveraging Spring Boot for the backend REST API, MySQL for data persistence, and a modern HTM-

L/JavaScript frontend with Chart.js for visualization. The platform integrates Ollama as a local AI inference engine and APKTool for Android APK decompilation.

30 Figure 1 presents the layered architecture of the platform, organized into four distinct tiers: the Presentation Layer (frontend SPA), the Business Logic Layer (Spring Boot 3.2 / Java 17), the AI and Analysis Layer (OllamaService, CodeAnalyzerService, ApkAnalyzerService), and the Data Layer (MySQL 8.0, apktool.jar, Ollama LLaMA 3.2).

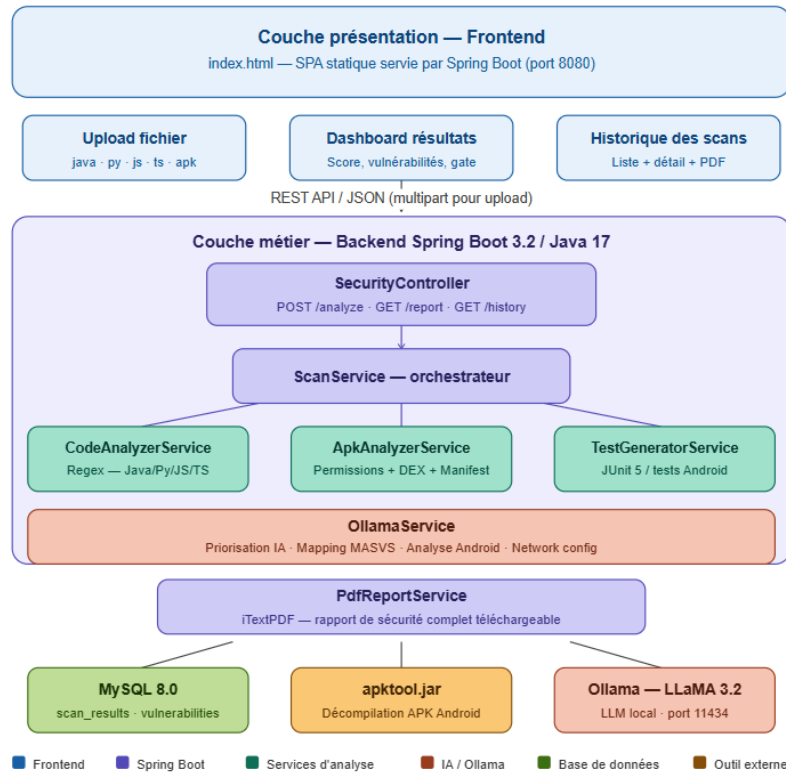


Figure 1: SecTest Generator — Layered software architecture showing the four tiers: Presentation (Frontend), Business Logic (Spring Boot), Analysis Services, and Data Layer (MySQL, APKTool, Ollama LLaMA 3.2)

35 The architecture consists of the following layers:

- **Presentation Layer:** A single-page web application (`index.html`) served statically by Spring Boot on port 8080, providing three modules: file upload (Java/Py/JS/TS/APK), results dashboard (score, vulnerabilities, gate), and scan history (list + detail + PDF).
- 40 • **Business Logic Layer:** The `SecurityController` exposes REST endpoints (`POST /analyze`, `GET /report`, `GET /history`). The `ScanService` orchestrates the full analysis pipeline.
- 45 • **Analysis Services Layer:** `CodeAnalyzerService` applies regex-based rules across Java/Python/JS/TS; `ApkAnalyzerService` leverages APK-Tool for manifest decompilation and DEX inspection; `TestGeneratorService` produces JUnit 5 tests; `OllamaService` handles AI enrichment; `PdfReportService` generates downloadable security reports via iTextPDF.
- 50 • **Data Layer:** MySQL 8.0 persists scan results and vulnerabilities; APK-Tool (`apktool.jar`) handles APK decompilation; Ollama LLaMA 3.2 runs locally on port 11434 as the AI inference engine.

Figure 2 illustrates the complete analysis pipeline data flow.

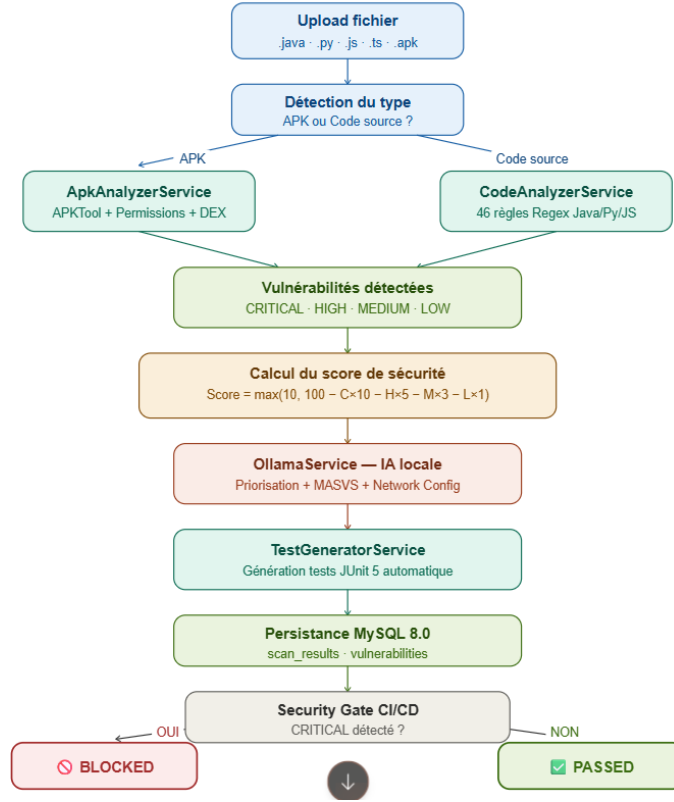


Figure 2: SecTest Generator — Analysis pipeline data flow: file upload, type detection (.apk vs. source code), vulnerability analysis, scoring, AI enrichment via Ollama, test generation, MySQL persistence, and Security Gate decision (PASSED / BLOCKED)

2.2. Software Functionalities

SecTest Generator offers the following key functionalities:

1. **Multi-language Source Code Analysis:** Detection of security vulnerabilities in Java (20 rules), Python (12 rules), and JavaScript/TypeScript (14 rules) using regex-based pattern matching covering CWE categories including SQL Injection (CWE-89), Command Injection (CWE-78), Hard-coded Secrets (CWE-798), XXE Injection (CWE-611), and Insecure Deserialization (CWE-502).
2. **Android APK Analysis:** Integration with APKTool [8] for manifest de-

compilation, enabling detection of: dangerous permissions (CRITICAL/HIGH-/MEDIUM severity), exported components without permissions (CWE-926), cleartext traffic (CWE-319), debug mode activation (CWE-215), and insecure storage patterns.

- 65 3. **Automatic JUnit Test Generation:** For each detected vulnerability, the platform generates an executable JUnit 5 test that verifies the absence of the vulnerability pattern, enabling regression prevention in CI/CD pipelines.
- 70 4. **AI-Powered Analysis:** Ollama llama3.2 [9] enriches analysis results with: vulnerability prioritization, MASVS/MASTG control mapping for Android findings, exported component risk analysis, and Network Security Config recommendations.
- 75 5. **Security Gate CI/CD:** A binary gate decision (BLOCKED/PASSED) based on the presence of CRITICAL vulnerabilities, supporting integration into automated deployment pipelines.
6. **Security Score Calculation:** A quantitative score computed as:

$$Score = \max(10, 100 - 10 \times C - 5 \times H - 3 \times M - 1 \times L)$$

where C , H , M , L represent counts of CRITICAL, HIGH, MEDIUM, and LOW vulnerabilities respectively. The minimum guaranteed score is **10**, ensuring meaningful differentiation even for highly vulnerable targets.

- 80 7. **PDF Report Generation:** Professional PDF reports generated with iText library, containing vulnerability details, recommendations, generated tests, and Security Gate status.
- 85 8. **Analytics Dashboard:** Chart.js-powered visualizations including severity distribution (doughnut chart), category breakdown (bar chart), score evolution (line chart), and file type distribution (pie chart).

3. Illustrative Examples

3.1. Scenario 1: Source Code Analysis — Java Vulnerable Application

We analyzed `VulnerableApp.java`, a Java class intentionally containing multiple security anti-patterns including hardcoded credentials, SQL query concatenation, runtime command execution, MD5/SHA1 hash usage, insecure de-
 90 catenation, runtime command execution, MD5/SHA1 hash usage, insecure deserialization, and XXE-vulnerable XML parsing.

Vulnérabilités détectées						15 trouvée(s)
SÉVÉRITÉ	VULNÉRABILITÉ	CWE	LIGNE	CODE DÉTECTÉ	RECOMMANDATION	
HIGH	Hardcoded Password	CWE-793	L. 9	<code>password = "password"</code>	Utilisez <code>System.getenv("DB_PASSWORD")</code> ou un vault de secrets...	
HIGH	Hardcoded API Key/Token	CWE-793	L. 10	<code>apiKey = "key"</code>	Utilisez <code>System.getenv("API_KEY")</code> ou un gestionnaire de secrets...	
HIGH	Hardcoded API Key/Token	CWE-793	L. 11	<code>apiKey = "key"</code>	Utilisez <code>System.getenv("API_KEY")</code> ou un gestionnaire de secrets...	
CRITICAL	SQL Injection	CWE-89	L. 16	<code>stmt.executeQuery("SELECT ... " + userInput)</code>	Utilisez <code>PreparedStatement</code> : <code>stmt.setString(1, userInput)</code> ...	
CRITICAL	Command Injection	CWE-78	L. 21	<code>Runtime.getRuntime().exec(userInput)</code>	N'utilisez jamais de données utilisateur dans <code>exec()</code> . Utilisez une liste d'argum...	
CRITICAL	Command Injection	CWE-78	L. 22	<code>Runtime.getRuntime().exec(userInput)</code>	N'utilisez jamais de données utilisateur dans <code>exec()</code> . Utilisez une liste d'argum...	
HIGH	Weak Hash MD5	CWE-323	L. 27	<code>MessageDigest md = MessageDigest.getInstance("MD5")</code>	Utilisez SHA-256 ou <code>bcrypt</code> : <code>MessageDigest.getInstance("SHA-256")</code> ...	
HIGH	Weak Hash SHA1	CWE-323	L. 28	<code>MessageDigest md = MessageDigest.getInstance("SHA1")</code>	Utilisez SHA-256 minimum : <code>MessageDigest.getInstance("SHA-256")</code> ...	
CRITICAL	Insecure Deserialization	CWE-502	L. 35	<code>ObjectInputStream ois = new ObjectInputStream(inputStream)</code>	N'utilisez jamais <code>readObject()</code> sur des données non fiables. Utilisez <code>JSON</code> ...	
CRITICAL	Insecure Deserialization	CWE-502	L. 36	<code>Object o = ois.readObject()</code>	N'utilisez jamais <code>readObject()</code> sur des données non fiables. Utilisez <code>JSON</code> ...	
HIGH	Path Traversal	CWE-22	L. 41	<code>File f = new File(filePath)</code>	Validez <code>path.normalize().startsWith(queuePath)</code> ...	
MEDIUM	Insecure Random	CWE-339	L. 47	<code>SecureRandom rng = new SecureRandom()</code>	Utilisez <code>SecureRandom</code> : <code>new SecureRandom()</code> ...	
MEDIUM	Insecure Random	CWE-339	L. 48	<code>SecureRandom rng = new SecureRandom()</code>	Utilisez <code>SecureRandom</code> : <code>new SecureRandom()</code> ...	
LOW	printStackTrace	CWE-549	L. 56	<code>e.printStackTrace()</code>	Utilisez un logger : <code>logger.error("Erreur", e)</code> ...	
CRITICAL	XXE Injection	CWE-611	L. 63	<code>java.xml.parsers.DocumentBuilderFactory.newInstance()</code>	Désactivez les entités : <code>factory.setFeature("http://apache.org/xml/features/secu...</code>	

Figure 3: Scenario 1 — Vulnerability detection results for `VulnerableApp.java`: 15 vulnerabilities, Security Gate BLOCKED

The analysis detected **15 vulnerabilities**: 6 CRITICAL, 6 HIGH, 2 MEDIUM, and 1 LOW, resulting in a security score of **10/100** (minimum guaranteed) and a **BLOCKED** Security Gate status. Table 1 summarizes the critical findings.

Table 1: Critical vulnerabilities detected in VulnerableApp.java

Vulnerability	Severity	CWE	Line
SQL Injection	CRITICAL	CWE-89	L.16
Command Injection	CRITICAL	CWE-78	L.21–22
Insecure Deserialization	CRITICAL	CWE-502	L.35–36
XXE Injection	CRITICAL	CWE-611	L.63
Hardcoded Password	HIGH	CWE-798	L.9
Hardcoded API Key	HIGH	CWE-798	L.10–11
Weak Hash MD5	HIGH	CWE-328	L.27
Path Traversal	HIGH	CWE-22	L.41
Insecure Random	MEDIUM	CWE-338	L.47–48
printStackTrace	LOW	CWE-209	L.56

```

Tests JUnit Générés — Powered by Ollama IA

// =====
// AUTO-GENERATED SECURITY TESTS
// Généré par Security Regression Test Generator
// Fichier analysé: vul.java
// NE PAS MODIFIER MANUELLEMENT
// =====

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.DisplayName;
import static org.junit.jupiter.api.Assertions.*;
import java.util.regex.*;
import java.io.*;

```

Figure 4: Scenario 1 — Severity distribution chart and KPI metrics

95 The Ollama AI module produced the following prioritization: (1) Command Injection — fix immediately, (2) SQL Injection — fix immediately, (3) Hardcoded Password — fix this sprint, (4) Hardcoded API Key — fix this sprint. MASVS mapping: Hardcoded Password → MASVS-CODE-4, Hardcoded API Key → MASVS-STORAGE-1.

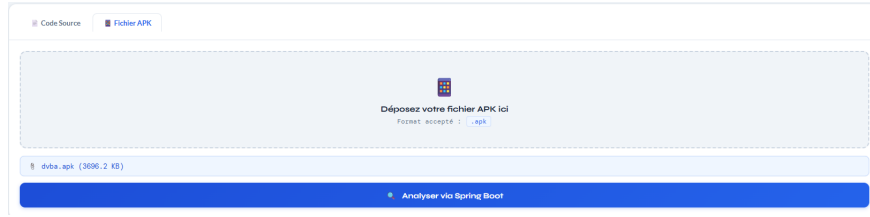


Figure 5: Scenario 1 — Generated JUnit tests with AI prioritization and MASVS mapping

3.2. Scenario 2: Android APK Analysis — InsecureBankv2

The second scenario uses InsecureBankv2 [5], a deliberately vulnerable Android banking application. The APK was decompiled with APKTool 3.0.2.

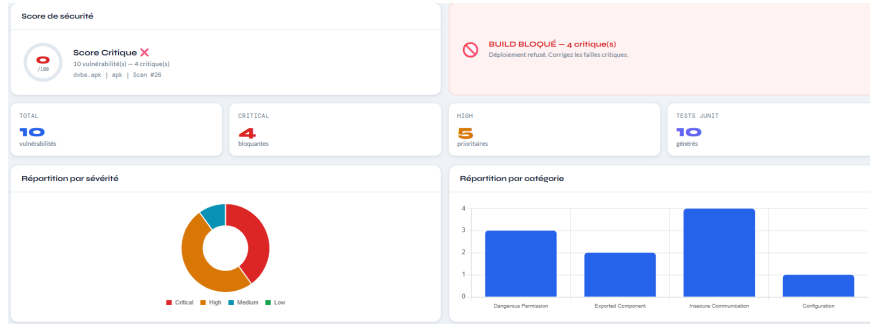


Figure 6: Scenario 2 — APK analysis results for InsecureBankv2: 19 vulnerabilities, score 10/100, BLOCKED

The analysis revealed **19 vulnerabilities**: 9 CRITICAL, 8 HIGH, and 2 MEDIUM, yielding a score of **10/100** with a **BLOCKED** gate. Key findings are listed in Table 2.

Table 2: Critical findings in InsecureBankv2.apk

Vulnerability	Severity	CWE	Component
Exported: PostLogin	CRITICAL	CWE-926	Activity
Exported: DoTransfer	CRITICAL	CWE-926	Activity
Exported: ViewStatement	CRITICAL	CWE-926	Activity
Exported: TrackUserContentProvider	CRITICAL	CWE-926	Provider
Exported: MyBroadCastReceiver	CRITICAL	CWE-926	Receiver
Exported: ChangePassword	CRITICAL	CWE-926	Activity
Mode Debug activated	CRITICAL	CWE-215	Manifest
Permission SEND_SMS	CRITICAL	CWE-250	Manifest
Permission READ_PHONE_STATE	CRITICAL	CWE-250	Manifest
HTTP Insecure	HIGH	CWE-319	Manifest
Backup enabled	HIGH	CWE-530	Manifest

Sévérité	Vulnérabilité	CWE	Ligne	Code détecté	Recommandation
HIGH	Permission dangereuse: USE_BIOMETRIC	CWE-250	L. 7	uses-permission: android.permission.USE_BIOMETRIC	Vérifiez si USE_BIOMETRIC est strictement nécessaire...
HIGH	Permission dangereuse: USE_FINGERPRINT	CWE-250	L. 7	uses-permission: android.permission.USE_FINGERPRINT	Vérifiez si USE_FINGERPRINT est strictement nécessaire...
MEDIUM	Permission dangereuse: INTERNET	CWE-250	L. 7	uses-permission: android.permission.INTERNET	Vérifiez si INTERNET est strictement nécessaire...
CRITICAL	Composant exporté sans permission: SendMoney	CWE-926	L. 22	activity android:exported	Ajoutez android.permission ou mettez android:exported="false"...
CRITICAL	Composant exporté sans permission: ViewBalance	CWE-926	L. 27	activity android:exported	Ajoutez android.permission ou mettez android:exported="false"...
HIGH	HTTP non sécurisé	CWE-319	L. 2	network http android:networkSecurityConfig	Utilisez HTTPS pour toutes les communications réseau...
CRITICAL	Cleartext Traffic autorisé	CWE-319	L. 6	android:usesCleartextTraffic	Désactivez android:usesCleartextTraffic="false"...
HIGH	Backup activé	CWE-530	L. 6	android:allowBackup	Désactivez android:allowBackup="false"...
HIGH	HTTP non sécurisé	CWE-319	L. 3	network http android:networkSecurityConfig	Utilisez HTTPS pour toutes les communications réseau...
CRITICAL	Cleartext Traffic autorisé	CWE-319	L. 3	android:usesCleartextTraffic	Désactivez android:usesCleartextTraffic="false"...

Figure 7: Scenario 2 — Exported components detection with MASVS Android mapping by Ollama AI

3.3. Scenario 3: Android APK Analysis — DVBA

The third scenario uses DVBA [6] to validate platform generalizability.

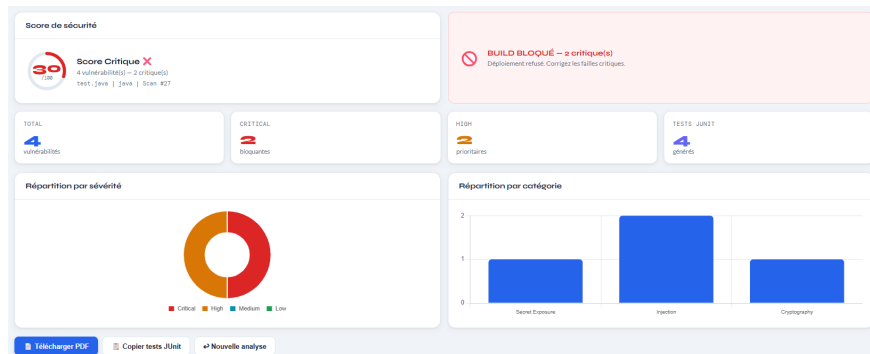


Figure 8: Scenario 3 — DVBA APK analysis: cleartext traffic and exported components detected

Key findings: **SendMoney** and **ViewBalance** exported without permission (CWE-926), cleartext traffic (CWE-319), and biometric permissions **USE_BIOMETRIC**/**USE_FINGERPRINT** (CWE-250). The Ollama AI generated a Network Security Config recommendation:

Listing 1: Recommended network_security_config.xml generated by Ollama AI

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <network-security-config>
115    <base-config cleartextTrafficPermitted="false">
4      <trust-anchors>
5        <certificates src="system"/>
6      </trust-anchors>
7    </base-config>
120 </network-security-config>

```

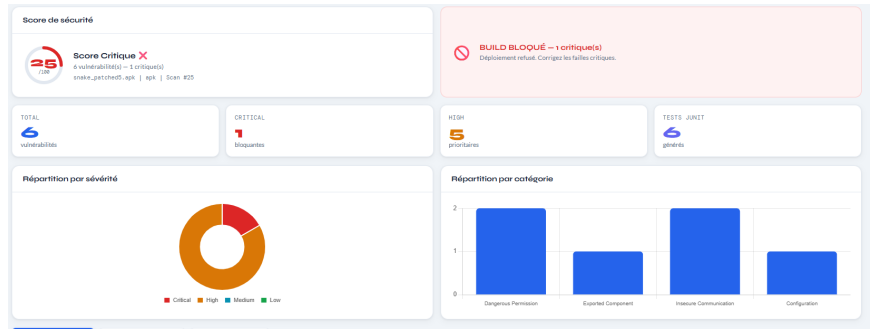
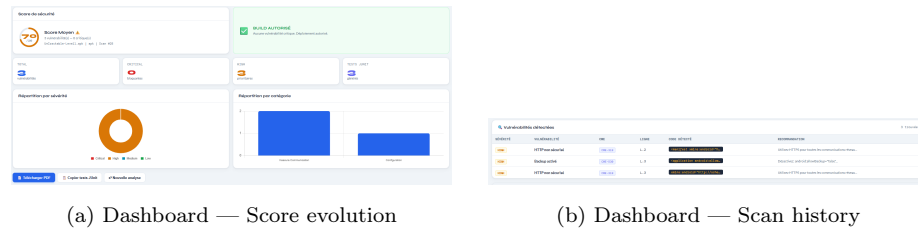


Figure 9: Scenario 3 — JUnit tests generated for DVBA with AI prioritization and MASVS mapping

3.4. Scenario 4: Analytics Dashboard and PDF Report



(a) Dashboard — Score evolution

(b) Dashboard — Scan history

Figure 10: Scenario 4 — Analytics Dashboard: scan history and security metrics

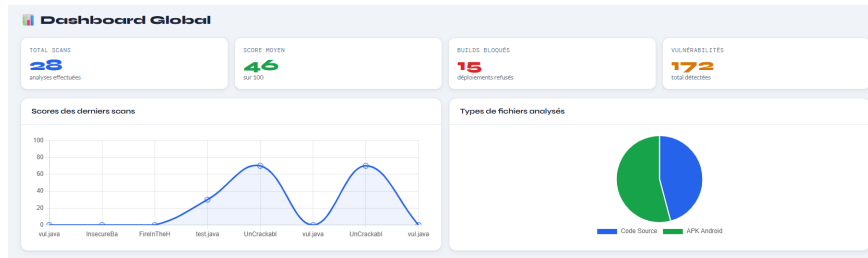


Figure 11: Scenario 4 — Generated PDF security report with vulnerability details and JUnit tests

4. Impact

SecTest Generator addresses a critical gap in the DevSecOps toolchain by
 125 combining static analysis, AI-powered enrichment, and automated test genera-

tion in a unified self-hosted workflow.

Security Regression Prevention: The auto-generated JUnit tests integrate directly into Maven/Gradle build configurations, ensuring corrected vulnerabilities cannot regress into production. The Security Gate aligns with shift-left security principles.

Android Security Coverage: The MASVS/MASTG mapping addresses Chapters 14 and 15 of the MASTG, covering Labs 4 and 5. The exported component detection covers a frequently overlooked attack surface [7].

AI-Enhanced Analysis: By leveraging a local LLM (Ollama llama3.2), the platform provides context-aware analysis without transmitting sensitive source code to external services, addressing data privacy concerns in enterprise environments.

4.1. Comparison with MobSF and Existing Tools

Mobile Security Framework (MobSF) [11] is the most widely used open-source tool for Android APK security analysis. Table 3 provides a detailed comparison between SecTest Generator and MobSF across key dimensions relevant to DevSecOps pipelines.

Table 3: Detailed comparison between SecTest Generator and MobSF

Feature	SecTest Generator	MobSF
<i>Analysis Capabilities</i>		
Android APK Analysis		
Source Code Analysis (Java/Py/JS)		–
Manifest decompilation (APKTool)		
Exported component detection		
Dangerous permissions detection		
Cleartext traffic detection		
Dynamic analysis	–	
<i>AI & Intelligence</i>		
AI-powered prioritization	(Ollama local)	–
MASVS/MASTG mapping (AI)		Partial
Finding → Test synthesis (AI)		–
Network Security Config generation		–
<i>DevSecOps Integration</i>		
JUnit 5 test generation		–
Security Gate CI/CD		–
REST API		
PDF Report		
Security score (quantitative)		Partial
<i>Privacy & Deployment</i>		
Local AI (no data leakage)		–
Self-hosted		
Spring Boot backend		–
Docker support	Planned	
<i>Usability</i>		
Web dashboard		
Analytics charts		Partial
Scan history		
Open source	14	

As shown in Table 3, SecTest Generator differentiates itself from MobSF in three key areas: (1) **Multi-language source code analysis** — MobSF focuses exclusively on mobile applications (APK/IPA), while SecTest Generator also analyzes Java, Python, and JavaScript source code; (2) **AI-powered test synthesis** — SecTest Generator uniquely generates executable JUnit 5 regression tests from each security finding using Ollama LLaMA 3.2, a capability absent from MobSF; and (3) **CI/CD Security Gate** — SecTest Generator provides a binary BLOCKED/PASSED decision with a quantitative security score, enabling direct pipeline integration. MobSF’s strength lies in dynamic analysis capabilities, which SecTest Generator does not currently support and plans to address in future work.

Table 4: Broader comparison of SecTest Generator with security tools

Feature	SecTest	MobSF	SonarQube	JADX	SpotBugs	AndroBugs
Source Code Analysis		—		—		—
APK Analysis			—		—	
JUnit Test Generation		—	—	—	—	—
MASVS Mapping		Partial	—	—	—	—
AI Prioritization		—	—	—	—	—
Security Gate CI/CD		—		—		—
Local AI (Privacy)		—	—	—	—	—
Dynamic Analysis	—		—	—	—	—
PDF Report				—	—	
Open Source						

5. Quality Assurance

Detection Accuracy: Against InsecureBankv2, the platform detected all 9 CRITICAL vulnerabilities and 8 HIGH vulnerabilities (recall = 100% for known critical findings). Against DVBA, all exported components and cleartext traffic were correctly identified.

Analysis Performance: Source code analysis (<100 lines) completes in
 160 under 2 seconds. APK analysis with APKTool averages 15–30 seconds. AI
 enrichment via Ollama adds 30–60 seconds on CPU-only systems.

Rule Coverage:

Table 5: Security rule coverage by language and category

Language	Rules	CRITICAL	HIGH	Categories
Java	20	5	8	Injection, Crypto, Storage, XSS
Python	12	4	5	Injection, Deserialization, Crypto
JavaScript	14	5	6	Injection, XSS, Auth, SSRF
APK Android	21	5	9	Permissions, Export, Network

Score Formula Validation: With the updated formula $\max(10, 100 - 10C - 5H - 3M - L)$, the minimum guaranteed score is 10. For InsecureBankv2 (9
 165 CRITICAL, 8 HIGH, 2 MEDIUM): $\max(10, 100 - 90 - 40 - 6) = \max(10, -36) =$
 10.

Limitations: Regex-based analysis may produce false positives in generated
 or framework code. Ollama response time is hardware-dependent. Generated
 JUnit tests are unit-level (static); instrumentation tests for runtime Android
 170 behavior are planned for future work.

6. Conclusions

This paper presented SecTest Generator, an AI-powered security regression
 test generation platform for DevSecOps pipelines. The platform combines static
 analysis (46 rules for source code, 21 rules for APK), local AI inference via
 175 Ollama LLaMA 3.2, APKTool-based Android manifest analysis, and automated
 JUnit test generation.

Evaluations on InsecureBankv2 and DVBA demonstrate effectiveness in de-
 tecting critical Android vulnerabilities with automatic MASVS/MASTG map-
 ping. Compared to MobSF, SecTest Generator uniquely provides JUnit regres-

180 sion test generation, CI/CD Security Gate integration, and local AI-powered
analysis without data leakage.

Future work includes: PHP/C language support, Android instrumentation
test generation, AI response latency optimization, and CVE-based dependency
scanning.

185 SecTest Generator is publicly available at [https://github.com/fatimaezzahra123-wq/](https://github.com/fatimaezzahra123-wq/SecTest)
SecTest under the MIT License.

Acknowledgements

The authors thank the National School of Applied Sciences, Cadi Ayyad
University, for supporting this research.

190 References

- [1] IBM Security. Cost of a Data Breach Report 2023. IBM Corporation, 2023.
- [2] J. Humble and D. Farley. Continuous Delivery. Addison-Wesley, 2010.
- [3] SonarSource. SonarQube Documentation. <https://www.sonarsource.com/sonarqube/>, 2025.
- 195 [4] OWASP. Mobile Application Security Verification Standard (MASVS).
<https://mas.owasp.org/MASVS/>, 2024.
- [5] D. Shetty. Android-InsecureBankv2. [https://github.com/](https://github.com/dineshshetty/Android-InsecureBankv2)
dineshshetty/Android-InsecureBankv2, 2023.
- [6] Damn Vulnerable Bank App. [https://github.com/rewanthtammana/](https://github.com/rewanthtammana/Damn-Vulnerable-Bank)
200 Damn-Vulnerable-Bank, 2023.
- [7] A. P. Felt et al. Android permissions demystified. ACM CCS, pp. 627–638,
2011.
- [8] C. Tumbleson. APKTool. <https://apktool.org/>, 2025.

- [9] Ollama. Run large language models locally. <https://ollama.com/>, 2025.
- 205 [10] VMware. Spring Boot Documentation. <https://spring.io/projects/spring-boot>, 2025.
- [11] MobSF Team. Mobile Security Framework (MobSF). <https://mobsf.github.io/Mobile-Security-Framework-MobSF/>, 2025.